UNIVERSIDADE FEDERAL DE ITAJUBÁ

ECOX21 - MARATONA DE PROGRAMAÇÃO I

Profs. Edmilson Marmo e Luiz Olmes



Warmup: Aula 10

21/10/2026

Regras

1. Há 3 problemas que devem ser resolvidos no tempo estipulado.
2. Os dados de entrada devem ser lidos a partir da entrada padrão.
3. Os dados de saída devem ser escritos na saída padrão.
4. Quando uma linha contém vários valores, eles estarão separados por um único espaço. Não há espaços extras na entrada/saída. Não há linhas em branco na entrada.
5. A entrada e a saída usam o alfabeto latino. Não haverá letras com til, acentos, tremas ou outros sinais diacríticos.
6. Todas as linhas da entrada e da saída, incluindo a última contêm o caractere de fim de linha.
7. O código fonte de cada solução deve ser enviado pelo Boca: `boca.unifei.edu.br`

Ambiente de Testes

A correção das soluções enviadas é realizada no sistema operacional Red Hat Enterprise Linux, versão 8.6 (Ootpa), usando os seguintes compiladores/interpretadores:

```
C: gcc versão 8.5.0 20210514 (Red Hat 8.5.0-10)
C++: g++ versão 8.5.0 20210514 (Red Hat 8.5.0-10)
Java: openjdk versão 1.8.0_342
Python: python3 versão 3.6.8
```

Limites

```
Memória (C, C++, Python): 1GB
Memória (Java): 1GB + 100MB stack
Tamanho máximo do código fonte: 100KB
Tamanho máximo do arquivo executável: 1MB
```

Códigos que extrapolem os limites permitidos receberão *Runtime Error* como resposta.

Comandos de Compilação

```
C: gcc -g -O2 -std=gnu11 -static -lm
C++: g++ -g -O2 -std=gnu++17 -static -lm
Java: javac
```

Códigos C/C++

- O programa deve retornar zero, executando, como último comando, `return 0` ou `exit(0)`.
- Para entradas grandes, objetos `iostream` podem ser lentos, devido a questões de sincronização de buffer com a biblioteca `stdio`. Recomenda-se desabilitar este mecanismo de sincronização em programas que empreagam `std::cin` e `std::cout` através dos seguintes comandos:

  ```
  std::ios::sync_with_stdio(0);
  std::cin.tie(0);
  ```

  Note que, neste caso, deve-se evitar usar `printf` e `scanf` no mesmo programa, pois a separação dos buffers pode levar a resultados inesperados.

Códigos Java

- O programa não deve estar encapsulado em um `package`.
- Para cada problema, o arquivo `.java` e a `public class` devem ter o mesmo nome `basename` mostrado no Boca.
- Comando de execução: `java -Xms1024m -Xmx1024m -Xss100m`

Códigos Python

- Apenas Python 3 é suportado. Python 3 não é compatível com Python 2.
- **Atenção**: não é garantido que soluções escritas em Python executarão dentro do tempo limite especificado para cada problema.
- Comando de execução: `python3`

Problema $\mathcal{A}$

GUERRA

Timelimit: 1.0s

Uma guerra está sendo travada entre dois países, A e B. Como um cidadão leal do país C, você decide ajudar na espionagem do seu país participando das negociações de paz que estão ocorrendo atualmente (incógnito, é claro). Há N pessoas nas negociações (não incluindo você), mas você não sabe qual pessoa pertence a qual país. Você pode ver as pessoas conversando entre si e, ao observar o comportamento delas durante suas ocasionais conversas individuais, pode adivinhar se são amigas ou inimigas. Na verdade, o que seu país precisaria saber é se determinados pares de pessoas são do mesmo país ou se são inimigos. Você pode receber tais perguntas do governo de C mesmo durante as negociações de paz, e deve dar respostas com base em suas observações até o momento. Felizmente ninguém fala com você, já que ninguém presta atenção à sua humilde presença. Mais formalmente, considere seguintes operações:

- `setFriends`(x, y): mostra que x e y são do mesmo país
- `setEnemies`(x, y): mostra que x e y são de países diferentes
- `areFriends`(x, y): retorna verdadeiro se você tem certeza de que x e y são amigos
- `areEnemies`(x, y): retorna verdadeiro se você tem certeza de que x e y são inimigos

As duas primeiras operações devem sinalizar um erro se contradizerem seu conhecimento anterior. As duas relações '`Friends`' (denotada por $\sim$) e '`Enemies`' (denotada por $*$) têm as seguintes propriedades:

$\sim$ é uma relação de equivalência, ou seja:

1. Se $x \sim y$ e $y \sim z$ então $x \sim z$ (Os amigos dos meus amigos também são meus amigos).
2. Se $x \sim y$ então $y \sim x$ (A amizade é mútua).
3. $x \sim x$ (Todo mundo é amigo de si mesmo).

$*$ é simétrica e irreflexiva:

1. Se $x * y$ então $y * x$ (O ódio é mútuo).
2. Não $x * x$ (Ninguém é inimigo de si mesmo).

Por fim:

1. Se $x * y$ e $y * z$ então $x \sim z$ (Um inimigo comum torna duas pessoas amigas).
2. Se $x \sim y$ e $y * z$ então $x * z$ (Um inimigo de um amigo é um inimigo).

As operações `setFriends`(x, y) e `setEnemies`(x, y) devem preservar essas propriedades.

Entrada

A primeira linha contém um único inteiro, N ($N \leq 10000$), o número de pessoas. Cada uma das linhas seguintes contém uma tripla de inteiros, cxy, onde c é o código da operação:

- $c = 1$, `setFriends`
- $c = 2$, `setEnemies`
- $c = 3$, `areFriends`
- $c = 4$, `areEnemies`

e x e y são seus parâmetros, que são inteiros no intervalo $[0, N)$, identificando duas pessoas (diferentes). A última linha contém apenas zeros: '`0 0 0`'.

Saída

Para cada operação `areFriends` e `areEnemies`, escreva '`0`' (significando não) ou '`1`' (significando sim) na saída. Além disso, para cada operação `setFriends` ou `setEnemies` que contradiga o conhecimento anterior, imprima um '`-1`' na saída; note que tal operação não deve produzir nenhum outro efeito e a execução deve continuar. Um `setFriends` ou `setEnemies` bem-sucedido não gera saída.

Exemplos

Entrada	*Saída*
10 1 0 1 1 1 2 2 0 5 3 0 2 3 8 9 4 1 5 4 1 2 4 8 9 1 8 9 1 5 2 3 5 2 0 0 0	1 0 1 0 0 -1 0

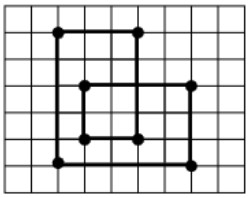
Problema $\mathcal{B}$

CORTES A LASER

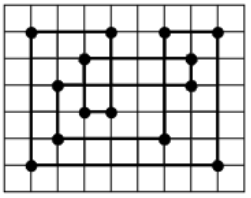
Timelimit: 0.1s

Uma máquina de corte a laser para chapas de madeira possui um cabeçote de laser que pode movimentar-se apenas nas direções horizontal e vertical.

Um dos testes realizados para verificar seu funcionamento consiste em programar a máquina para realizar uma sequência não vazia de cortes consecutivos que inicia e termina sempre no mesmo ponto. Cada corte na sequência, exceto o primeiro, inicia no ponto em que o corte anterior terminou. Nenhum corte toca a borda da chapa a ser cortada. As figuras (a) e (b) abaixo mostram dois exemplos de sequência de cortes, respectivamente com 8 e 14 cortes.

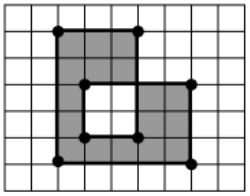


(a)

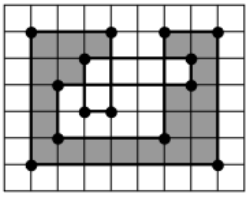


(b)

O seu chefe pediu para que você determine a área do pedaço de maior área produzido pela sequência de cortes, desconsiderando o pedaço que contém as bordas da chapa cortada. Isto é, devem ser considerados apenas os pedaços contidos no polígono definido pelas linhas de corte. As figuras (c) e (d) abaixo mostram respectivamente os maiores pedaços produzidos pelos cortes das figuras (a) e (b).

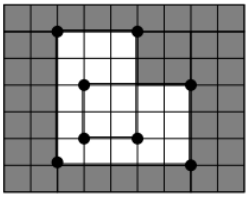


(c)

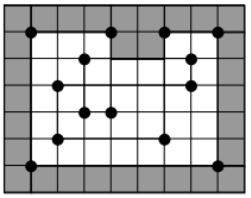


(d)

Para ilustrar, as figuras (e) e (f) abaixo mostram o pedaço descartado (que contém as bordas da chapa cortada) dos cortes das figuras (a) e (b), respectivamente.



(e)



(f)

Entrada

A primeira linha da entrada contém um inteiro N ($4 \leq N \leq 10^4$), indicando o número de cortes da sequência. A segunda linha contém dois inteiros X_0 e Y_0 que são a posição inicial do cabeçote de

laser ($1 \leq X_0, Y_0 \leq 10^3$). Cada uma das N linhas seguintes contém dois inteiros X_i e Y_i indicando a coordenada de um corte ($1 \leq X_i, Y_i \leq 10^3$). Todas as coordenadas da entrada são distintas entre si, exceto a primeira e a última.

Saída

Imprima um inteiro indicando a área do maior pedaço produzido pela sequência de cortes.

Exemplos

Entrada	*Saída*
8 2 1 7 1 7 4 3 4 3 2 5 2 5 6 2 6 2 1	17

Entrada	*Saída*
14 1 1 8 1 8 6 6 6 6 2 2 2 2 4 7 4 7 5 3 5 3 3 4 3 4 6 1 6 1 1	21

Problema C

OS SUSPEITOS

Timelimit: 0.1s

A Covid-19 é uma variação da Síndrome Respiratória Aguda Grave (*Severe Acute Respiratory Syndrome* - SARS), uma pneumonia atípica de etiologia desconhecida que foi reconhecida como uma ameaça global em meados de março de 2003. Para minimizar a transmissão a outras pessoas, a melhor estratégia é separar os suspeitos dos outros.

Na Unifei, existem muitos grupos de estudantes. Estes grupos se relacionam com os mais diversos aspectos: esportes, projetos, monitorias, etc. Os alunos de um mesmo grupo têm contato frequente com os demais, e um mesmo aluno pode fazer parte de vários grupos. Para evitar possíveis transmissões de Covid, a Unifei mantém listas de membros de todos os grupos de estudantes e estabelece a seguinte regra em seu procedimento operacional padrão: "*Quando um membro de um grupo é suspeito, todos os membros do grupo são suspeitos*".

No entanto, os dirigentes da Unifei sabem que não é fácil identificar todos os suspeitos quando um aluno é oficialmente reconhecido como suspeito. Assim, a Unifei solicita a sua ajuda, com um programa para identificar os suspeitos da infecção.

Entrada

A entrada contém vários casos. Cada caso de teste começa com dois inteiros N e M, indicando, respectivamente, o número de alunos e o número de grupos ($0 \leq N \leq 30000; 0 \leq M \leq 500$). Cada aluno é identificado por um inteiro único entre 0 e $N-1$ e, inicialmente, o aluno 0 é reconhecido como suspeito. Seguem M linhas, cada uma descrevendo um grupo. Cada linha começa com um inteiro K, representando o número de membros daquele grupo, seguido por K inteiros, representando os alunos do grupo. Valores $N = M = 0$ indicam o fim da entrada.

Saída

Para cada caso de teste, seu programa deve produzir uma única linha contendo o número de alunos suspeitos de infecção.

Exemplos

Entrada	*Saída*
100 4 2 1 2 5 10 13 11 12 14 2 0 1 2 99 2 200 2 1 5 5 1 2 3 4 5 1 0 0 0	4 1 1